

Taxi ERP

Code Architecture

Defines the structural blueprint of every class in the Taxi ERP platform — frontend TypeScript components, hooks, services, and PDF templates, alongside backend PostgreSQL entity tables, trigger functions, and RPC routines. Each entry specifies its stereotype, responsibility, public surface, and dependency direction. No implementation code; structure only.

PARADIGM Mixed TypeScript (SPA) + SQL (PostgREST routines, triggers, RLS policies)

SCOPE Frontend SPA classes · Backend / DB layer classes · ~30 high-level classes total

NOTATION Structured block per class: name · stereotype · responsibility · members · dependencies · used by

SOURCE Derived from Taxi ERP System Design Document v1.0

Table of Contents

1. Architecture Overview	2
1.1 Layered Topology	2
1.2 Panel Dependency Direction	2
1.3 Module Coupling Principles	3
2. Frontend Class Hierarchy	3
3. Backend / DB Class Hierarchy	4
4. Class Definitions	5
4.1 Frontend Classes	5
4.2 Backend / DB Classes	9
5. Dependency Direction Rules	14
5.1 Allowed dependencies	14
5.2 Forbidden dependencies	14
5.3 Soft references for optional cross-panel links	15

1. Architecture Overview

1.1 Layered Topology

The Taxi ERP codebase is partitioned across four runtime layers. Each layer owns a distinct responsibility set and may only depend on the layer immediately below it. This strict layering is what allows individual panels (Master, Daily Work, Accounts, Reports) to be added or removed without rupturing the rest of the system. The browser layer is the only surface a user ever touches; the edge layer terminates TLS and applies rate-limiting; the VM layer hosts the entire self-hosted Supabase stack inside a single Docker Compose topology; and the external layer provides off-VM backup, CI/CD, and monitoring sidecars. No layer reaches upward except via response data, so the dependency graph is acyclic and auditable in a single glance.

Layer	Hosts	Inbound from	Outbound to
Browser	React SPA (Vite build): components, hooks, Zustand stores, TanStack Query cache, React Hook Form + Zod, @react-pdf/renderer templates	Operator only	Cloudflare Edge over HTTPS
Edge	Cloudflare Tunnel (cloudflared); TLS termination, rate-limit, bot mitigation, optional Zero Trust email-OTP SSO	Browser	VM localhost:8000 (Kong gateway)
VM (Docker Compose)	Kong gateway, PostgREST, GoTrue, Storage API, PostgreSQL 16, postgres-meta, Supabase Studio, Uptime Kuma, cloudflared sidecar	Edge	PostgreSQL (internal); Backblaze B2 (egress for backups)
External	Backblaze B2 (encrypted pg_dump archives), GitHub Actions (CI/CD), Uptime Kuma alert webhooks (Discord/email)	VM (egress only)	Operator laptop (key holder for age decryption)

Table 1.1 - Layered topology and permitted dependency direction.

1.2 Panel Dependency Direction

The four functional panels are layered exactly like their underlying database schemas. The Master panel is the foundation: every other panel reads its reference data (customers, vehicles, rates, GST config, document sequences). The Daily Work panel produces operational data (duty slips, bills) by consuming Master reference data. The Accounts panel reads bill and duty-slip rows to auto-post ledger entries; it never writes back upstream. The Reports panel is purely read-only - its views aggregate across all lower panels but produce no persistent writes. This strict layering is enforced at three levels: PostgreSQL schema namespaces (CREATE SCHEMA master, operations, billing, accounts, reports), foreign-key direction (no FK ever points upward), and SPA route registration (each panel module is lazy-loaded and may be code-split out without touching the others).

Panel	Schema	Reads from	Writes to
Master	master.*	core.* (companies, user_profiles)	master.* only

Panel	Schema	Reads from	Writes to
Daily Work	operations.*, billing.*	master.* (rates, vehicles, customers)	operations.duty_slips, billing.bills, billing.bill_duty_slips
Accounts	accounts.*	billing.bills, operations.duty_slips	accounts.ledger_entries only
Reports	reports.* (views)	billing.*, operations.*, accounts.*, master.*	Nothing (read-only views)

Table 1.2 - Per-panel permitted data dependencies.

1.3 Module Coupling Principles

Three principles keep the panels decoupled enough to be removed individually. First, each panel owns its own PostgreSQL schema, so DROP SCHEMA billing CASCADE removes every trace of the Billing panel without touching master or operations. Second, hard foreign keys are reserved for mandatory links (customer_id, vehicle_id, company_id); optional cross-panel links use nullable bigint columns validated at the application layer, so a future Driver Management panel can be removed without leaving broken FK references in operations.duty_slips. Third, the SPA registers each panel as a lazily-loaded route module behind a feature flag, so disabling a panel is a one-line configuration change with no recompile of the others. Together these principles make the system genuinely modular - not just architecturally layered, but operationally detachable.

2. Frontend Class Hierarchy

The frontend is a React 18 + Vite 5 single-page application. The hierarchy below shows composition (parent owns child) via indentation; siblings are at the same level. Cross-cutting providers wrap the entire app, the router dispatches to lazy-loaded panel modules, and each panel module groups its own pages, forms, and shared components. Service classes (Supabase client, query-key registry, PDF template factory) sit alongside the component tree and are injected via hooks.

App	<> - root mount
└─ AuthProvider	<> - GoTrue session, JWT claims, role
└─ RoleGuard	<> - gates UI by user_role claim
└─ AppRouter	<> - lazy panel routes behind feature flags
└─ MasterPanelModule	<> - /master/* routes
└─ DailyWorkPanelModule	<> - /daily-work/* routes
└─ AccountsPanelModule	<> - /accounts/* routes
└─ ReportsPanelModule	<> - /reports/* routes
└─ DutySlipForm	<> - shared by Daily Work panel
└─ BillingForm	<> - shared by Daily Work panel

└─ LedgerBookView	◁ - Accounts panel
└─ ReportTableView	◁ - TanStack Table v8 wrapper, shared by all 3 reports
└─ useAuth	◁ - reads AuthProvider context
└─ useEntityQuery	◁ - TanStack Query wrapper per entity
└─ supabaseClient	◁ - typed PostgREST + Storage client
└─ PdfTemplateFactory	◁ - @react-pdf/renderer template registry

Figure 2.1 - Frontend composition tree. Indentation = ownership; arrows show child direction.

3. Backend / DB Class Hierarchy

The backend is a self-hosted Supabase stack: PostgreSQL 16 with six logical schemas, PostgREST exposing each table as a REST resource, GoTrue for JWT issuance, and Storage for company logos. The hierarchy below lists the entity classes (each maps 1:1 to a table or view) grouped by schema, followed by the SQL routines (triggers and RPC functions) that operate on those entities. Trigger functions attach to a single table; RPC functions are called explicitly from the SPA. RLS helper functions are pure-SQL utilities used inside every policy definition.

Database	◁ - PostgreSQL 16 instance
└─ core schema	◁ - tenant boundary and identity
└─ Company	◁ - core.companies (1 row per deployment)
└─ UserProfile	◁ - core.user_profiles (1:1 with auth.users)
└─ master schema	◁ - reference data, no transactions
└─ Customer	◁ - master.customers (company + personal client types)
└─ VehicleCluster	◁ - master.vehicles + master.vehicle_groups + master.vehicle_types
└─ Rate	◁ - master.rates (per customer/vehicle/duty_type triple)
└─ GstConfig	◁ - master.gst_config (inter/intra-state rates)
└─ DocumentSequence	◁ - master.document_sequences (duty slip + bill numbering)
└─ operations schema	◁ - daily transactional data
└─ DutySlip	◁ - operations.duty_slips (atomic operational unit)
└─ billing schema	◁ - invoice aggregation
└─ Bill	◁ - billing.bills + billing.bill_duty_slips (junction)
└─ accounts schema	◁ - ledger entries
└─ LedgerEntry	◁ - accounts.ledger_entries (single source of financial truth)
└─ system schema	◁ - audit + settings

└─ AuditLog	<> - system.audit_log (append-only, JSONB snapshots)
└─ Setting	<> - system.settings (per-company key/value config)
└─ Database Routines	<> - PL/pgSQL functions, triggers, RPCs
└─ FnAssignDutySlipNo	<> - BEFORE INSERT on operations.duty_slips
└─ FnCalculateGst	<> - BEFORE INSERT on billing.bills
└─ FnSetInterstate	<> - BEFORE INSERT/UPDATE on master.gst_config
└─ FnAuditRow	<> - generic, attached to every business table
└─ GenerateBill	<> - POST /rest/v1/rpc/generate_bill
└─ CurrentCompanyContext	<> - current_company_id() + current_user_role()

Figure 3.1 - Backend composition tree. Schemas own entities; routines attach to entities.

4. Class Definitions

Each class is documented in a structured block: name, stereotype (the architectural role the class plays), one-line responsibility, public members (names only - no types and no implementation), the classes it depends on, and the classes that depend on it. The block is the contract; the implementation is left to the developer. No code snippets appear in this document - by design.

4.1 Frontend Classes

App <>	
Responsibility	Root mount point. Wraps the entire SPA in AuthProvider and AppRouter; renders the active panel module.
Public members	root, routerOutlet, errorBoundary
Dependencies	AuthProvider, AppRouter
Used by	main.tsx (Vite entrypoint)

AuthProvider <>	
Responsibility	Holds the GoTrue session, decodes JWT claims (company_id, user_role), exposes sign-in / sign-out, refreshes tokens on focus.
Public members	session, user, role, companyId, signIn, signOut, refresh
Dependencies	supabaseClient, useAuth
Used by	App, RoleGuard, every panel module

RoleGuard

Responsibility	Conditional renderer. Hides children if the current user_role is not in the allowed list; redirects to /unauthorized on hard violations.
Public members	allowedRoles, fallback, children
Dependencies	useAuth
Used by	Panel route elements, action buttons

AppRouter

Responsibility	Route table. Maps URLs to lazy-loaded panel modules; gates each route behind RoleGuard and a feature-flag check.
Public members	routes, navigate, currentPanel
Dependencies	MasterPanelModule, DailyWorkPanelModule, AccountsPanelModule, ReportsPanelModule, RoleGuard
Used by	App

MasterPanelModule

Responsibility	Code-split bundle for the Master panel. Mounts Company Detail, Utilities (Vehicles + Doc No. Control), Customer, Rate Management, and GST Management pages.
Public members	routes, navigationItems, featureFlag
Dependencies	supabaseClient, useEntityQuery, RoleGuard
Used by	AppRouter

DailyWorkPanelModule

Responsibility	Code-split bundle for the Daily Work panel. Mounts Duty Slip, Billing, Change/Cancel Bill, and Print Bill/Duty Slip pages.
Public members	routes, navigationItems, featureFlag
Dependencies	supabaseClient, useEntityQuery, DutySlipForm, BillingForm, PdfTemplateFactory
Used by	AppRouter

AccountsPanelModule

Responsibility	Code-split bundle for the Accounts panel. Mounts the Ledger Book page and the Record Receipt / Record Payment forms.
Public members	routes, navigationItems, featureFlag
Dependencies	supabaseClient, useEntityQuery, LedgerBookView
Used by	AppRouter

ReportsPanelModule

Responsibility	Code-split bundle for the Reports panel. Mounts Bill Cover, Bill Register, and Duty Register reports - all read-only.
Public members	routes, navigationItems, featureFlag
Dependencies	supabaseClient, useEntityQuery, ReportTableView, PdfTemplateFactory
Used by	AppRouter

DutySlipForm

Responsibility	Two-section form (booking info + duty info). Validates via Zod, computes total_km/total_hours client-side, looks up the rate, posts to operations.duty_slips. Triggers FnAssignDutySlipNo server-side.
Public members	customerId, vehicleId, dutyType, bookingFields, dutyFields, onSubmit, onCancel
Dependencies	useEntityQuery (customers, vehicles, rates), supabaseClient, PdfTemplateFactory (print)
Used by	DailyWorkPanelModule

BillingForm

Responsibility	Lists unbilled duty slips for a customer; lets the operator tick which to include and calls the GenerateBill RPC; shows the resulting bill row and offers Print.
Public members	customerId, selectedSlipIds, remarks, onGenerated
Dependencies	useEntityQuery (duty_slips, bills), supabaseClient.rpc(generate_bill), PdfTemplateFactory
Used by	DailyWorkPanelModule

LedgerBookView



Responsibility	Scrollable ledger table filtered by customer and date range; exposes Record Receipt / Record Payment buttons that open inline forms posting directly to accounts.ledger_entries.
Public members	filters, entries, runningBalance, onRecordReceipt, onRecordPayment
Dependencies	useEntityQuery (ledger_entries, customers, bills), supabaseClient
Used by	AccountsPanelModule

ReportTableView



Responsibility	Generic TanStack Table v8 wrapper used by all three reports. Accepts a column config and a query function; supports sort, filter, and a Print button that delegates to PdfTemplateFactory.
Public members	columns, queryFn, filters, onPrint
Dependencies	useEntityQuery, PdfTemplateFactory
Used by	ReportsPanelModule (Bill Cover, Bill Register, Duty Register)

useAuth



Responsibility	Thin wrapper over AuthProvider context. Returns the current session, user, role, and company id; throws if used outside AuthProvider.
Public members	session, user, role, companyId, isReady
Dependencies	AuthProvider
Used by	Every component that needs role/tenant context

useEntityQuery



Responsibility	Generic TanStack Query wrapper. Takes an entity name and optional filter, returns cached rows with auto-refetch on focus and mutation invalidation.
Public members	list(filter), get(id), create(payload), update(id, payload), delete(id), invalidate()
Dependencies	supabaseClient, queryKeyRegistry
Used by	Every panel page and form

supabaseClient		<>
Responsibility	Typed PostgREST + Storage client. Singleton instance; wraps supabase-js with typed table references and a per-entity query builder.	
Public members	from(table), rpc(name, args), storage.bucket(name), auth, channel(name)	
Dependencies	@supabase/supabase-js, queryKeyRegistry	
Used by	useEntityQuery, PdfTemplateFactory, AuthProvider	

PdfTemplateFactory		<>
Responsibility	Registry of @react-pdf/renderer templates: BillPDF, DutySlipPDF, BillCoverReportPDF, DutyRegisterReportPDF. Resolves a template by name, fetches joined data via TanStack Query, returns a blob URL for printing.	
Public members	render(templateName, data), getBlobURL(templateName, data), supportedTemplates	
Dependencies	supabaseClient, useEntityQuery (company, customer, vehicle joins)	
Used by	DailyWorkPanelModule, ReportsPanelModule, DutySlipForm, BillingForm	

4.2 Backend / DB Classes

Company		<>
Responsibility	Operator company profile. Tenant boundary for every other table. Usually one row per deployment; schema supports multiple for sister concerns.	
Public members	id, name, legal_name, gstin, state, address, logo_path, is_active	
Dependencies	none (root tenant)	
Used by	UserProfile, Customer, Vehicle, Rate, GstConfig, DutySlip, Bill, LedgerEntry, AuditLog, Setting, DocumentSequence	

UserProfile		<>
Responsibility	1:1 with auth.users. Holds the company_id (tenant) and role enum (owner / operator / accountant / viewer). Auto-created by a trigger on auth.users insert.	
Public members	id, company_id, full_name, role, is_active	
Dependencies	Company	
Used by	DutySlip (created_by), Bill (created_by, cancelled_by), LedgerEntry (created_by), AuditLog (changed_by)	

Customer <>	
Responsibility	Both B2B clients (regular bookings on credit) and one-time personal clients. client_type radio toggles required fields. State field drives GST lookup.
Public members	id, company_id, client_type, name, company_name, gstin, state, phone, is_active
Dependencies	Company
Used by	Rate, GstConfig, DutySlip, Bill, LedgerEntry

VehicleCluster <>	
Responsibility	Vehicle and its two orthogonal taxonomies (group = Sedan/SUV/Tempo; type = AC/Non-AC/Electric). Both taxonomies are user-defined lookups so the operator can extend them without code changes.
Public members	Vehicle: id, registration_no, vehicle_group_id, vehicle_type_id, rc_expiry, insurance_expiry, permit_expiry; VehicleGroup: id, name, display_order; VehicleType: id, name
Dependencies	Company
Used by	Rate (vehicle_group + vehicle_type), DutySlip (vehicle_id)

Rate <>	
Responsibility	Heart of the billing engine. Pins a (customer, vehicle_group, vehicle_type, duty_type) triple to a rate card. effective_from / effective_to enables time-travel - changing a rate inserts a new row and closes the previous one, preserving historical billing accuracy.
Public members	id, company_id, customer_id, vehicle_group_id, vehicle_type_id, duty_type, base_rate, per_km_rate, per_hour_rate, per_day_rate, extra_hour_rate, extra_km_rate, night_halt_rate, driver_allowance, min_charge, effective_from, effective_to
Dependencies	Company, Customer, VehicleGroup, VehicleType
Used by	DutySlip (rate_id)

GstConfig <>	
Responsibility	Per-customer GST rate card. is_interstate is derived automatically by FnSetInterstate comparing customer.state to company.state. Read by FnCalculateGst on bill insert to populate cgst/sgst/igst amounts.
Public members	id, company_id, customer_id, is_interstate, igst_rate, cgst_rate, sgst_rate, effective_from, effective_to
Dependencies	Company, Customer
Used by	Bill (gst_config_id), FnCalculateGst

DocumentSequence



Responsibility	Controls visible numbering of duty slips, bills, and future document types. mode field selects auto (trigger increments) vs manual (operator types, trigger validates uniqueness).
Public members	id, company_id, sequence_key, prefix, suffix, next_value, mode, padding_length, is_active
Dependencies	Company
Used by	FnAssignDutySlipNo, GenerateBill (bill_no assignment)

DutySlip



Responsibility	Atomic unit of operational data. Captures one assignment: customer, vehicle, duty start/end, opening/closing km, rate applied, extra charges, driver details. Bills aggregate one or more duty slips.
Public members	id, company_id, duty_slip_no, customer_id, vehicle_id, rate_id, duty_type, booking_date, guest_name, pickup_location, drop_location, duty_start_dt, duty_end_dt, opening_km, closing_km, total_km (generated), total_hours, base_amount, extra_*_amount, total_amount, custom_rate, status, bill_id, created_by
Dependencies	Company, Customer, Vehicle, Rate, UserProfile (created_by)
Used by	Bill (via bill_duty_slips junction), LedgerEntry (linked_duty_slip_id), FnAssignDutySlipNo, FnAuditRow

Bill



Responsibility	Numbered invoice covering one or more duty slips for a single customer. Totals are denormalized into bills and recomputed by triggers whenever a linked duty slip is added, removed, or modified. bill_duty_slips is the many-to-many junction with a partial unique index ensuring a duty slip is on at most one non-cancelled bill.
Public members	Bill: id, company_id, bill_no, bill_date, customer_id, gst_config_id, base_amount, extra_amount, total_before_tax, cgst_amount, sgst_amount, igst_amount, total_tax (generated), total_after_tax (generated), round_off, grand_total, status, cancelled_at, cancelled_by, created_by; BillDutySlip: bill_id, duty_slip_id, included_base, included_extra, included_total
Dependencies	Company, Customer, GstConfig, UserProfile, DutySlip (via junction)
Used by	LedgerEntry (linked_bill_id), FnCalculateGst, GenerateBill, FnAuditRow

LedgerEntry



Responsibility	Every financial movement in the system. entry_type enum classifies direction (sale / receipt / payment / adjustment / opening_balance). linked_bill_id and linked_duty_slip_id preserve traceability back to operational data.
Public members	id, company_id, entry_date, entry_type, customer_id, linked_bill_id, linked_duty_slip_id, debit_amount, credit_amount, narration, payment_mode, reference_no, created_by
Dependencies	Company, Customer, Bill, DutySlip, UserProfile
Used by	Reports Panel (all reports read this), FnAuditRow

AuditLog



Responsibility	Append-only history of every INSERT/UPDATE/DELETE on business tables. Old and new row snapshots stored as JSONB so the audit trail is self-describing even if the underlying schema evolves. Partitioned monthly by changed_at.
Public members	id, company_id, table_name, record_id, action, old_row (jsonb), new_row (jsonb), changed_by, changed_at
Dependencies	Company, UserProfile
Used by	Reports Panel (audit trail report, future); RLS denies all writes except via FnAuditRow

Setting



Responsibility	Per-company key/value configuration not significant enough for a column on core.companies. Stores things like default tax rate, preferred PDF font, GST reminder email template.
Public members	id, company_id, setting_key, setting_value, data_type, updated_at
Dependencies	Company
Used by	PdfTemplateFactory (font preferences), pg_cron GST reminder job

FnAssignDutySlipNo



Responsibility	BEFORE INSERT on operations.duty_slips. If duty_slip_no is NULL, fetches the next value from master.document_sequences (sequence_key = duty_slip), formats it with prefix + zero-pad + suffix, and updates next_value. If sequence row is missing, falls back to "DS-" + id.
Public members	NEW (trigger record), seq (document_sequences%ROWTYPE)
Dependencies	DocumentSequence
Used by	Attached as trg_duty_slip_no BEFORE INSERT ON operations.duty_slips

FnCalculateGst

Responsibility	BEFORE INSERT on billing.bills. Reads master.gst_config for (company_id, customer_id) where effective_to IS NULL. If is_interstate = true: igst_amount = (base+extra) * igst_rate / 100, cgst = sgst = 0. Else: cgst = sgst = (base+extra) * cgst_rate / 100, igst = 0. Also computes round_off to make grand_total a whole rupee.
Public members	NEW (trigger record), gst (gst_config%ROWTYPE)
Dependencies	GstConfig
Used by	Attached as BEFORE INSERT trigger on billing.bills; also fires during GenerateBill transaction

FnSetInterstate

Responsibility	BEFORE INSERT OR UPDATE on master.gst_config. Compares customer.state to company.state and sets NEW.is_interstate = (customer_state IS DISTINCT FROM company_state). The operator never sets is_interstate manually.
Public members	NEW (trigger record), customer_state, company_state
Dependencies	Customer (read state), Company (read state)
Used by	Attached as trg_gst_interstate BEFORE INSERT OR UPDATE ON master.gst_config

FnAuditRow

Responsibility	Generic trigger function attached to every business table (customers, vehicles, rates, gst_config, duty_slips, bills, ledger_entries, document_sequences). On INSERT/UPDATE/DELETE inserts a row into system.audit_log with table_name, record_id, action, old_row as JSONB, new_row as JSONB, and changed_by from auth.uid().
Public members	NEW, OLD, TG_TABLE_NAME, TG_OP
Dependencies	AuditLog (write target), auth.uid()
Used by	Attached to every business table; called automatically by PostgreSQL on every mutation

GenerateBill

Responsibility	POST /rest/v1/rpc/generate_bill. Single-transaction bill creation: (a) inserts billing.bills row (FnAssignDutySlipNo-equivalent assigns bill_no), (b) inserts billing.bill_duty_slips rows for each selected duty slip, (c) updates duty_slips.bill_id and duty_slips.status = billed, (d) fires FnCalculateGst to populate tax columns, (e) inserts a sale row into accounts.ledger_entries. Returns the complete bill row.
Public members	p_customer_id, p_duty_slip_ids[], p_remarks, p_bill_date
Dependencies	Bill, BillDutySlip (junction), DutySlip (update), GstConfig (via FnCalculateGst), LedgerEntry (auto-post sale), DocumentSequence (bill_no)
Used by	Called by BillingForm via supabaseClient.rpc()

CurrentCompanyContext	
Responsibility	Two STABLE SQL functions: current_company_id() returns NULLIF(auth.jwt() -> company_id, "")::bigint; current_user_role() returns COALESCE(auth.jwt() -> user_role, "viewer"). Used inside every RLS policy to enforce tenant isolation and role-gated writes without repeating JWT parsing logic.
Public members	current_company_id(), current_user_role()
Dependencies	auth.jwt() (Supabase Auth hook)
Used by	Every RLS policy on every tenant-scoped table

5. Dependency Direction Rules

The class hierarchy in this document obeys a small set of allowed-dependency rules. These rules exist so that any panel can be removed without cascading breakage. They are enforced at three levels: PostgreSQL schema and FK direction (database), PostgREST endpoint exposure (API), and SPA route registration (frontend).

5.1 Allowed dependencies

- UI component → Service (e.g. DutySlipForm → useEntityQuery → supabaseClient).
- UI component → Hook (e.g. DutySlipForm → useAuth for role check).
- Service → supabaseClient (single PostgREST entry point).
- Service → PdfTemplateFactory (for Print buttons).
- SQL Trigger → Entity (the trigger fires on one table; may read others).
- SQL RPC → multiple Entities (GenerateBill touches Bill, DutySlip, LedgerEntry in one transaction).
- Entity → Entity via FK (only downward in the panel hierarchy).
- Reports Panel → any Entity (read-only).

5.2 Forbidden dependencies

- Cross-panel UI calls (e.g. a Master page directly calling a Daily Work component). Panels communicate only via shared services and the database.
- Entity → upward Entity (e.g. master.rates cannot FK to operations.duty_slips). The hierarchy is acyclic.
- SQL Trigger writing to a different schema than its own table (except FnAuditRow, which is the only cross-schema writer, going to system.audit_log).
- Frontend code calling PostgreSQL directly - all DB access must go through PostgREST via supabaseClient.
- Reports Panel writing to any Entity - reports are views only; mutations belong to the source panel.
- Hard FK on optional cross-panel links (e.g. a future driver_id on duty_slips must be a nullable bigint validated at the application layer, not a FK).

- Direct mutation of system.audit_log from any client - only FnAuditRow may write to it; an RLS policy denies everything else.

5.3 Soft references for optional cross-panel links

When a panel-optional link is needed (e.g. a future Driver Management panel that links drivers to duty slips), the link must be implemented as a nullable bigint column without a FOREIGN KEY constraint. Validation moves to the application layer: the SPA checks the driver exists before save. This decouples the panels so the Driver panel can be removed without breaking operations.duty_slips (the column stays NULL). Hard FKs are reserved for mandatory links (customer_id, vehicle_id, company_id) where a missing parent would corrupt the data.